Module - 1

Introduction to operating system

→ operating system is a program.
↳ set of instructions

→ Operating system is a software
↳ Two types
⇐ 1) system software
⇐ 2) Application software (or) Application program

We can use the computer in an effective manner.
∄ user develops an application is called

→ operating system is a system software

Definition:- operating system acts as an interface between user of computer and computer hardware.

Goals of operating system:-

1) Execute user programs
2) Easy to use Computer
3) utilize computer hardware in effective manner.

Components of the Computer:-

Four Components of the computer
1) Hardware devices 2) OS
3) Application software 4) User.

From user view, Easy to use Computer is the major advantage of operating system.

From system view, Operating system acts as a resource allocator and control programs.

* An operating system is a program that manages the Computer hardware. It also provides a basis for application programs and acts as an intermediary b/w the computer user and the Computer hardware.

* It supports complex games, business applications and everything. It designed to be convenient, others to be efficient and others some combination of the two.

~~It designed to be convenient.~~

Components of a the Computer:-

A Computer system can be divided into four components.

1) The hardware
2) The operating system
3) The application programs
4) The users.

The hardware:- The central processing unit (CPU), the memory and the input/output (I/O) devices - provides the basic computing resources for the system.

The application programs: such as word processors, spreadsheets, compilers and web browsers. It defines the ways in which these resources are used to solve user's computing problems.

The users: The OS controls and coordinates the use of the hardware among the various application programs for the various users.

What Operating System Do:-

Depends on user view:-

1) For PC's OS is designed for convenience, easy of use and good performance. Don't care about resource utilization.

2) For mainframes, OS is designed for maximize resource utilization

3) For dedicated system such as workstation using shared resources from servers.

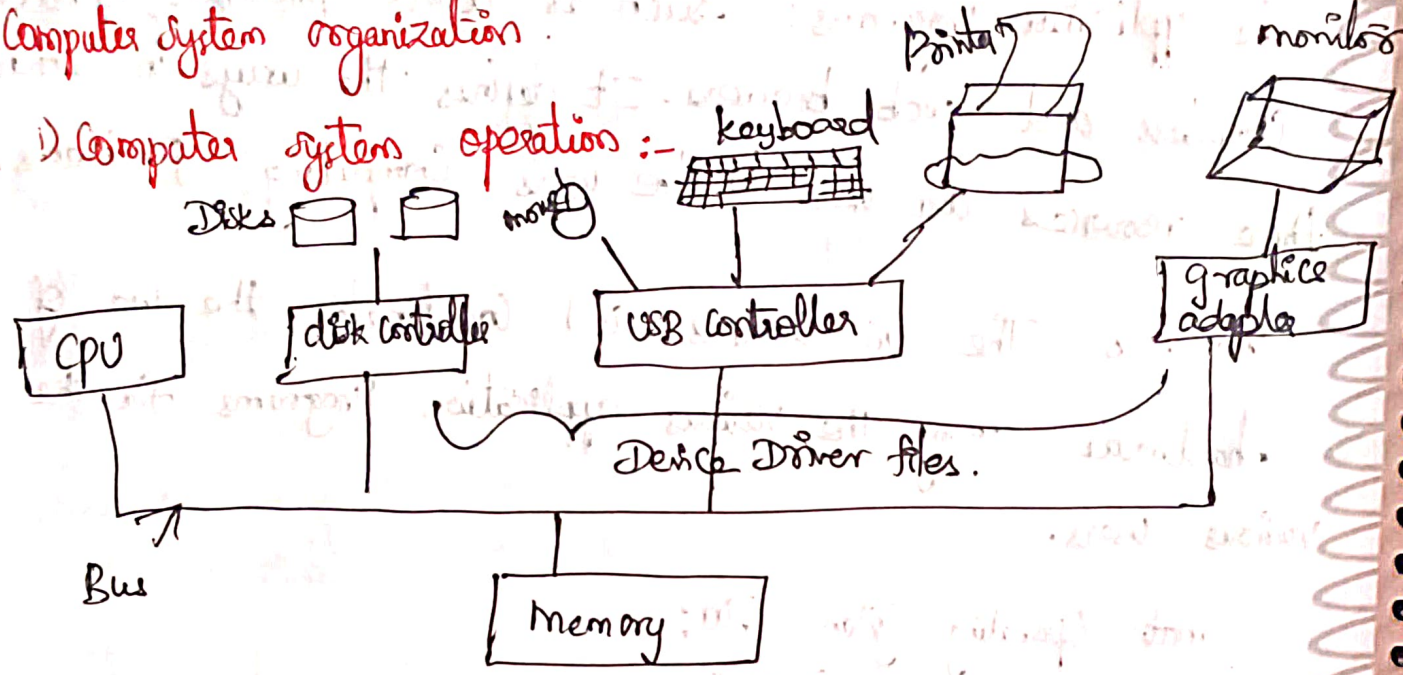
4) For handheld computers, OS is designed for better battery life.

Depends on a system view:

1) OS is a Resource Allocator
2) OS is a Control program.

Computer system organization

1) Computer system operation :-



* A modern computer system contains CPU and Device driver files on the memory which are connected through a common Bus.

* Disk controller controls the disks. Like USB controller controls i/o devices and graphics adapter controls monitor.

* Each and Every parts wants to send a signals through Bus.

* Every controller have local buffer. All the data can be stored in local buffer.

* Device controller and CPU can work concurrently.

* memory is a main memory and disk controller having disk it is a secondary memory. Disk data can be executed then send a signal to the CPU. That signal is called interrupt.

* once CPU receives an interrupt signals then all the data from local buffer to main memory. because CPU only executes the main memory program only.

* once execution over, again memory data can be transferred to the local buffer through CPU.

Bootstrap loader:- Computer to start running - for instance when it is powered up or rebooted it needs to have an initial program to run. This initial program are called bootstrap program. It is a simple program and it is stored in ROM (Read Only memory).

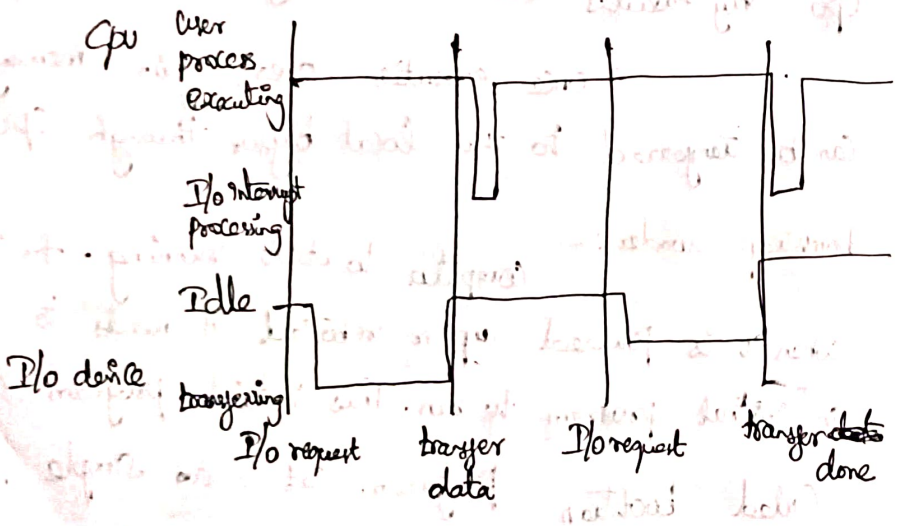
Program Counter:- It contains address of the next instructions to be executed.

CPU Control is shifted to the ISR (Interrupt Service Program) then ISR can be executed.

ISR is used to * Execution of ISR is transferring the data from memory to the local buffer or local buffer to the memory.

* Once ISR to be executed the cpu controls the next instructions (or) original program.

* Interrupt will be handle by explain this diagram:



2) Storage structure :-

In order to store the data, we have types of storage memories.

1) main memory (or) primary memory
2) Secondary memory
3) Cache memory.

1) main memory (or) primary memory :-

It is useful to store the data which can be used by CPU.

Eg:- RAM - Properties :-

Volatile m/y
↳ If s/m switch off it will be deleted.

2) Secondary memory :- It is used to store the data permentaly and it is used for backup purpose.

Eg: Hard disk - Properties :-

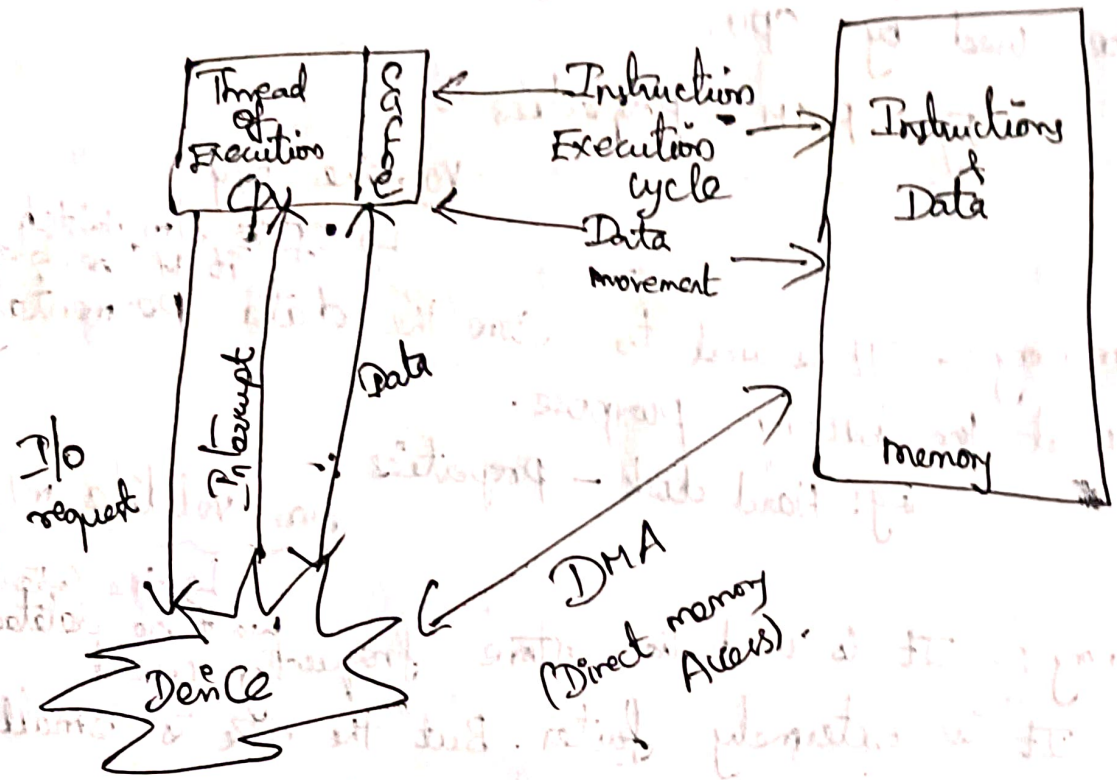
Non-volatile m/y.
↳ the content won't be deleted.

3) Cache memory :- It is used to store frequently used instructions. It is extremely faster. But the size is small.

Cache characteristics

	Storage capacity (or) size	Cost	Access speed.
1) Main m/y (or) primary m/y	1 GB, 2GB(n) 4GB	≃ 3k (expensive than secondary m/y).	very fast compare with secondary m/y.
2) Secondary m/y	1 TB	≃ 3k (Cheaper)	Slower
3) Cache m/y	1MB, 2MB(n) 8MB	≃ 3k (Expensive)	Frequently access so it is faster

3) I/O structure :-



Types of operating system :-

6 types of O/s are there. They are

1) Batch system
2) Multiprogramming System
3) Multitasking (or) Time shared system
4) Multiprocessor system
5) Distributed system
6) Real time operating system

1) Batch System :- A collection of Jobs. The users uses Punch Cards. The user provide all the informations in Punch Card and hand over to Computer Operator.

Assume (J1, J2, J3, J4, J5, J6, J7, J8, J9, J10) 10 Jobs. Computer operator prepare the batches for the given Jobs

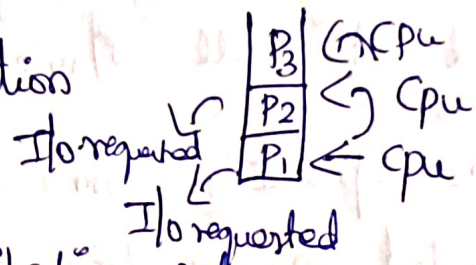
B1 B2

First B1 will be executed and get the output then B2 can be executed.

The major drawback is Cpu became Idle.

2) multiprogramming system :- multiprograms placed in a main m/y. and executed the multiprograms simultaneously.

Advantage :- Cpu utilization



3) Time sharing systems :- (multitasking system).

→ It is the extension of multiprogramming systems

→ Cpu allocates time to all the process

eg :- P1 P2 P3 [o/s allocates 2 msec for all these Process

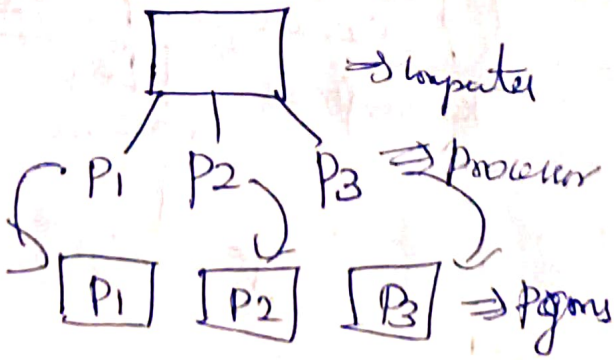
2msec 2msec 4msec

2msec

~~P3~~ ~~P2~~ ~~P1~~

2msec 2msec 2msec

4) multiprocessor system :- multi processor can be connected to the all processor.



Three advantages are there

1) Increased throughput
2) Economy (cost is very cheaper).
3) Reliability

5) Distributed System (or) network system :-

→ The same information can be shared (or) distributed to all the s/m's in the corresponding n/w.

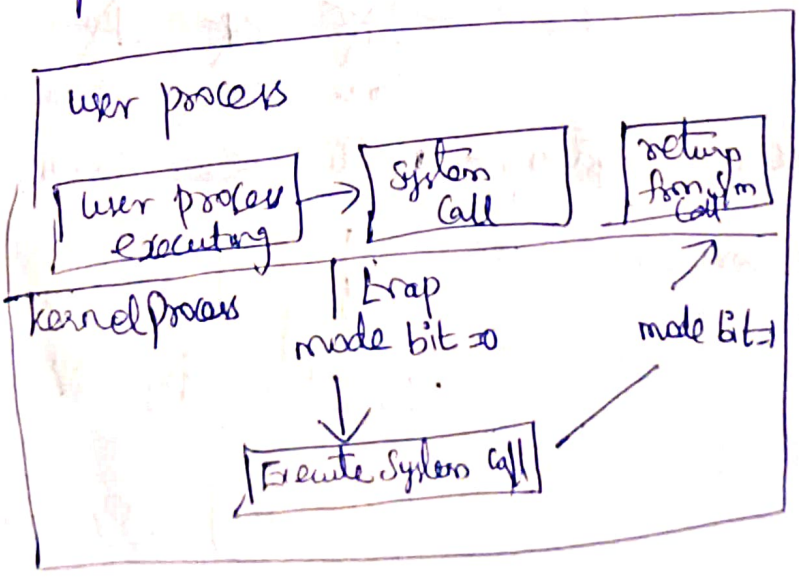
Eg :- Single project, all the modules are distributed to all s/m's

6) Real-time operating s/m's :- Real-time appln can be executed in this type and it can be executed by specific amount of time. 2 types are there.

Hard real-time s/m (The appln can be executed within a stimulated time). otherwise s/m fail

Soft real-time s/m (is any pblm then there is no pblm).

Operating system operations :-



→ Operating system is interrupt driven. 2 types of interrupt

1) H/w interrupt (The interrupt can be generated by I/O devices)
2) S/w interrupt (It is also called trap) eg: division/zero Exception.

→ Here we have dual mode, two types of operation

1) User mode
2) Kernel mode

To maintain modes, the hardware provides the mode bit. The mode bit values either (1 or 0). The mode bit is 0 then it is kernel mode and the mode bit is 1 then it is user mode. If the mode bit is 1 then it is user mode, so user mode means CPU will be executing user programs (ie) Appln pgms.

Functions of operating system (or) operating system components :-

There are five major functions of O/S.

They are

1) process management :-

Process means programs in execution. During execution the O/S loads the instructions from hard disk to main memory then it is called process.

Activities in process management :-

1) Create a process
2) Suspend a process
3) Resume a process
4) Handle process communications
5) Handle process synchronization
6) Deadlocks

2) Main memory management :-

main memory contains the execution of ~~the~~ programs by CPU. It is a volatile m/y

Actities for main m/y management.

1) During execution, O/S loads the program from ~~hard disk~~ into ~~main m/y~~ hard disk (Allocates the m/y)

2) Deallocates the m/y

Two approaches are there in allocating the m/y

1) Contigous m/y allocation

Two approaches.

1) MFT (multiprogramming Fixed Taxk)

2) MVT (multiprogramming Vaiable Task).

In older days use these two approaches

2) Non - contigous m/y allocation

2 approaches

1) paging

2) Segmentation

3) Secondary m/y management :-

The content of the m/y can be stored in permentaly. It is non-volatile m/y

Approaches

1) Contiguous allocation
2) Linked list allocation 3) Indexing allocation

4) File management :-

→ A collection of related information is called files.

→ File contains collections of bits, words, characters, bytes, bits.

Activities :-

1) Creating a file
2) Deleting a file
3) Reading / writing / ~~deleting~~ append / merging

5) I/O s/m management :-

→ Every I/O devices have device driver file

Eg :- printer.

Operating system services :

1) Program Execution :- O/s provides environment for the execution of the program.

→ During execution, O/s transfers the pgm from hard disk to main m/y

→ after execution, again it transfers the From main m/y to hard disk.

2) I/O operation :- when the pgm is in execution, the pgm may require I/O devices, o/s is responsible for the execution of the program.

3) File system manipulation :-

File system is a collection of directories. Operations of files are Read, write, open etc. o/s is responsible for file systems.

4) Communications :- process can be Communicate with each other. (ie) inter process Communication. It is implementation of message passing.

5) Error Detection :- o/s system everytime detects or monitor any error is occur or not. It checks wheather

→ cpu occur or not

→ m/y occur or not.

6) Resource Allocation :

o/s is responsible for allocate the resources for the process. then only process can be executed successfully. eg:- Hard disk, main memory, files etc

7) Accounting :- which person uses the which hardwares, (ie) security

8) Protection & Security :- o/s maintains the security and protect our data.

System Calls

→ System Calls is used in user's application program for the purpose of access the services of the O/s.

→ It is a part of operating system.
→ It is used to designing the operating system.
→ It is a programming interface to the services provided by OS
→ It is written in high level language C (or) C++
→ mostly accessed by programs via application Programming interface (API) rather than direct system call.
→ There are three major API's are there
1) Win 32 API for windows
2) Java API for Java Virtual Machine (JVM)
3) POSIX API for POSIX based O/s (UNIX, LINUX, APPLE)

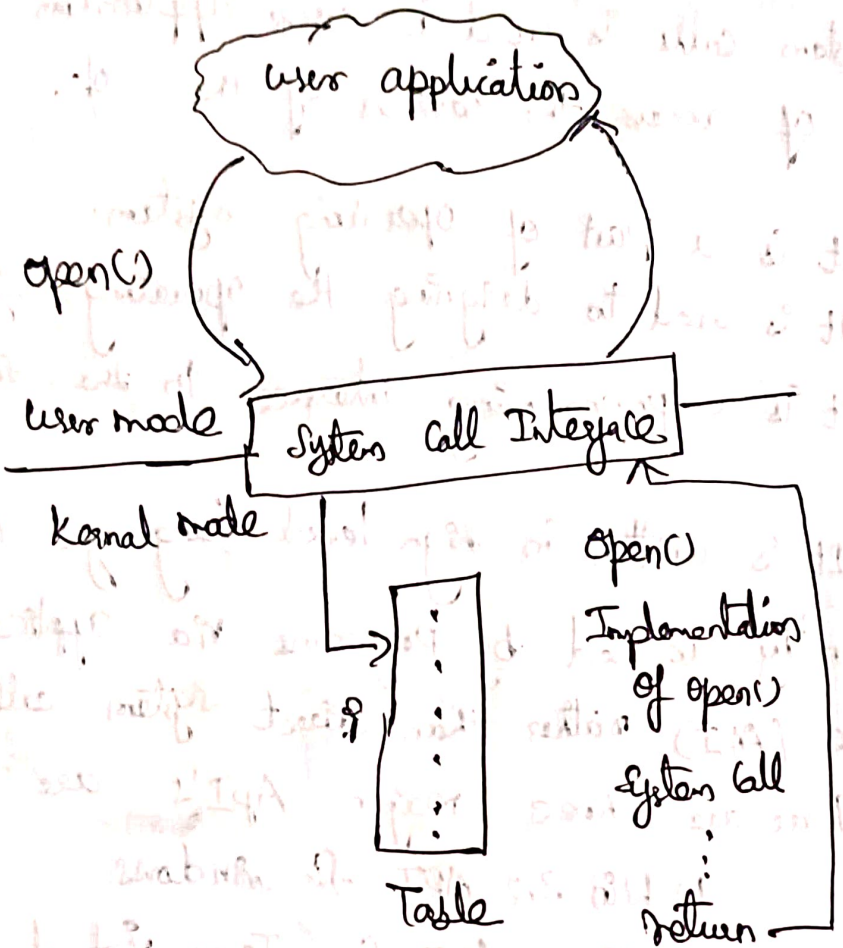
Example:- Standard API
→ Read() (It is available in stdio.h)
→ write()

~~direct read (file descriptor)~~ → Parameters.

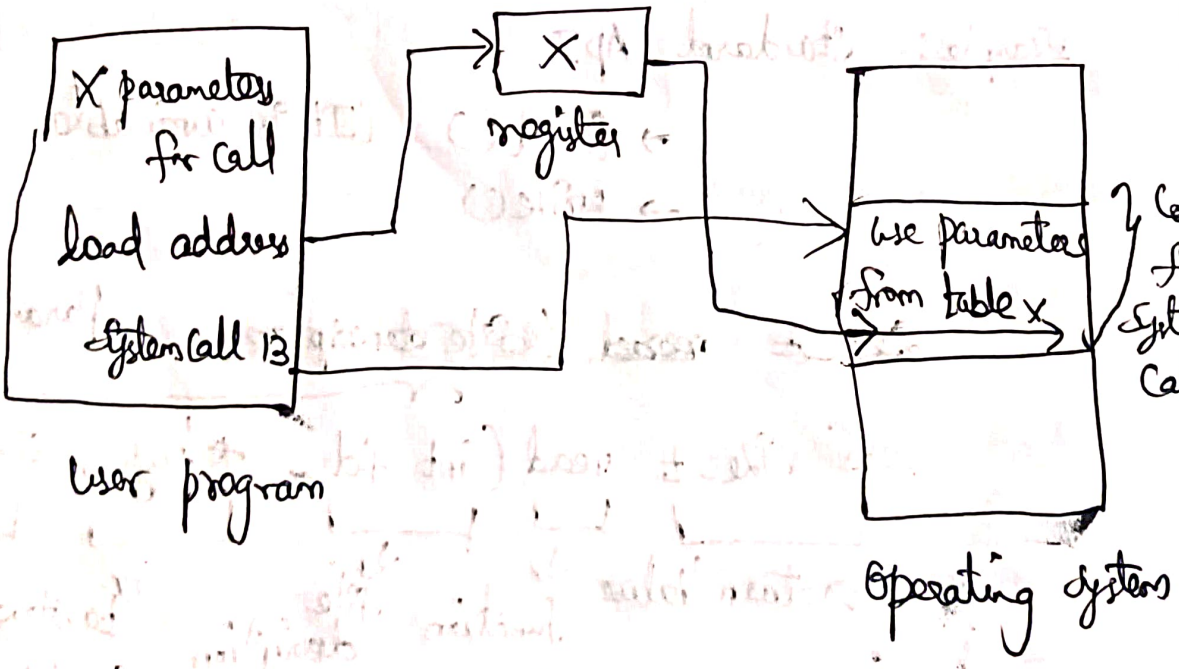
```
ssize_t read (int fd, *buf, size_t count)
```

- ssize_t → return value
- read → function name
- int fd → file description
- *buf → to store the data from source file
- size_t count → no. of bytes to be read

System Call Implementation



System Call Parameter Passing



Types of system calls in operating system

1) process control :- Create a process, terminate a process, load a process, Execute, wait, signal, allocate & free memory.

2) File management :- Create a file, delete, Open, read, write close.

3) Device management :- request device, release device, read, write

4) Intermediate maintainance :- get time or date, set time or date, Get system date, set system date, Get process file, device attributes, Set process file device attributes

5) Communication :- Create, delete Communicate Connects, Send, receive messages.

System Programs :-

Types of system Program

- → File management
- → Status information
- → File modification
- → Communication.
- → Programming language support
- → Program loading and execution

Definition :-

→ The most used s/m program is operating system.

→ It provides an environment for development and execution of the program.

→ It can interact with hardware devices (ie) hard disk, I/o devices etc

→ There are six types of system program in O/S.

1) File management s/m programs :-

It allows us to create a file, delete a file, copy a file, copy content of the file etc

2) Status information s/m program :-

It mainly provides the information about the system. Such as s/m date, s/m time and available m/y space (ie) harddisk space, it shows how many logons are in s/m's and how many files are there. The logged in user's informations will be displayed.

In windows Regedit it is an explorer to display the status of the ~~Operation~~ s/m.

3) File modification s/m program :-

~~A file modification~~

To change the content of the file.

Eg:- grep Command in linux

Programming language support :- These s/m pgms allows to Create various pgms like C, C++, JAVA, PYTHON, etc.

System pgms are Compilers, Assemblers & Interpreter.

- Compilers → High level to m/c code
- Assemblers → Assembly level to m/c code
- Interpreter → line by line execution

Some s/m Compilers are default with O/S but TURBO C, we have to install seperately. JAVA, Jdk to be installed

Programming loading and execution :-

O/S loads the data from harddisk into main memory that time loader s/m program is used.

Once loader can be executed then exec file can be created then the program can be ready to be execution.

Communication :- A process can communicate with another process b/w the virtual environment needed.

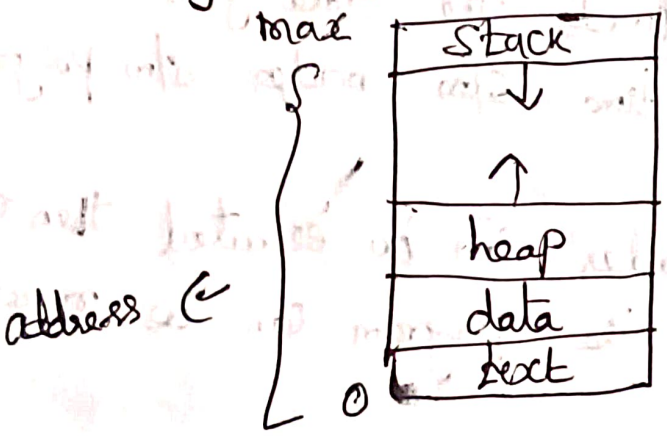
Process

Definition:- process is a program during execution.

↳ Collection of instructions to be do some specific tasks.

Eg:- C program execution.

→ Every process can be represented internally in main mfy.



Text segment :- It is also called as code segments. It is used to ~~Instruct~~ stores the instructions of the program.

Data segment :- It is used to store the global and static ~~local~~ variables.

Heap segment :- It is used to store dynamically memory allocated variables. (ie) run time variable allocation data

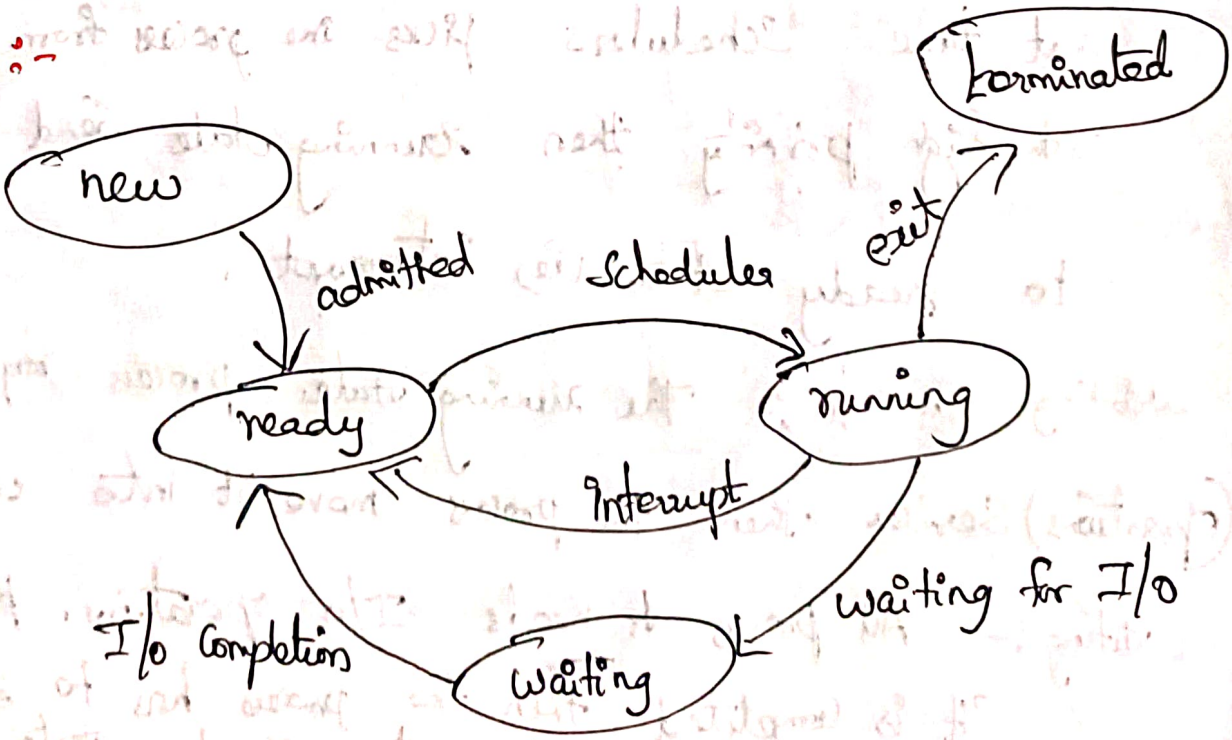
Stack Segment :- (or) Function stack.

→ It is used to store function parameters and local variable (ie) temporary variables and return variables.

Here stack segment can varies from one program to another program. It is travelled to downwards and heap segment can varies from one program to another program. It is travelled to upstairs.

Process states-

Diagram of process state (or) process state Transition Diagram :-



New :- A process is created is ~~called~~ in new state. (ie) a program is loaded into main memory by o/s then the process is in new state.

Ready :- The process is ready for execution. It is waiting for cpu. It contains a list of processes which are waiting to ready to be execution.

Scheduler :- The scheduler will picks from one process in ready state and allocates that process to be cpu.

Running :- cpu starts the execution in running state. cpu is executing the process.

Interrupt :- One process is executing in running state that time scheduler picks one process from ready state with high priority then running state send one signal to ready state (ie) interrupt.

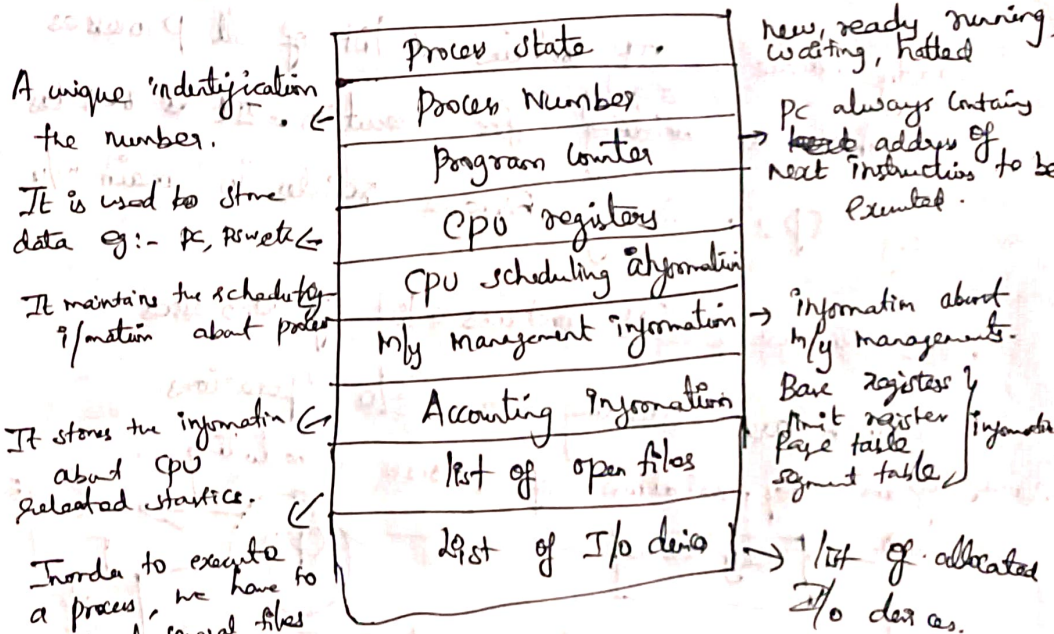
Waiting for I/O :- The running state process requires I/O (operations) services then the process move it into waiting state

Waiting :- All process needs I/O operations. ~~process~~ once it is completed then the process has to be moved from waiting to ready state.

Terminated :- All the process can be completed then it is moved from running to terminated. (ie) hard disk

Process Control Block (PCB) :-

Every process is represented in o/s by using PCB. Every process has its own PCB. It specifies the information about process.



Scheduling Queues

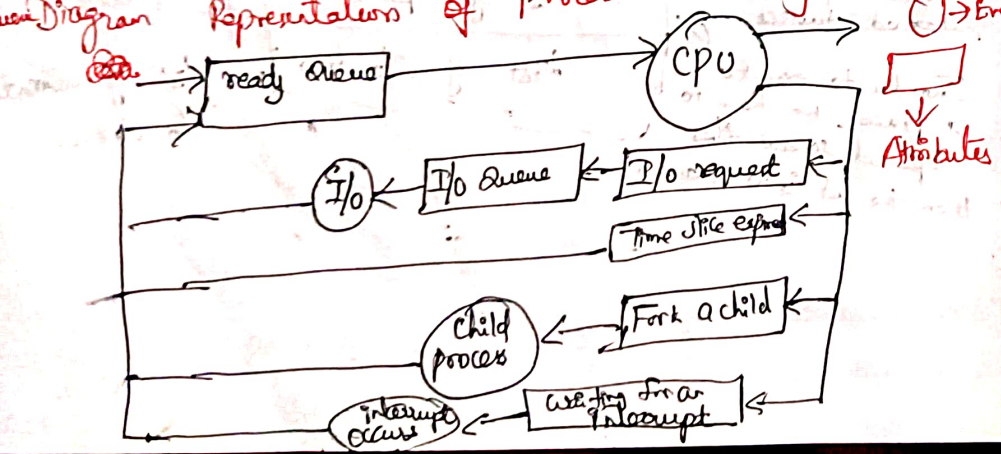
There are three types of Queues. They are Job Queue, Ready Queue and device Queue.

Job Queue :- It contains list of all the processes in the system. The content of Job Queue is stored in secondary m/y (ie) hard disk

Ready Queue :- It contains list of all processes which are ready for execution by CPU. Ready Queue resides in main m/y

Device Queue :- It contains a list of processes which are waiting for I/O operations.

Diagram Representation of Process scheduling :-



Types of schedulers :-

There are three types of schedulers. They are

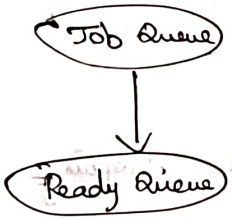
1) Long term schedulers (or) Job schedulers
2) short term schedulers (or) CPU schedulers
3) medium term schedulers.

Long term schedulers (or) Job schedulers :-

Long term schedulers transfer the data from secondary m/y to main m/y for execution.

In secondary m/y contains Job Queue
↳ list of processes

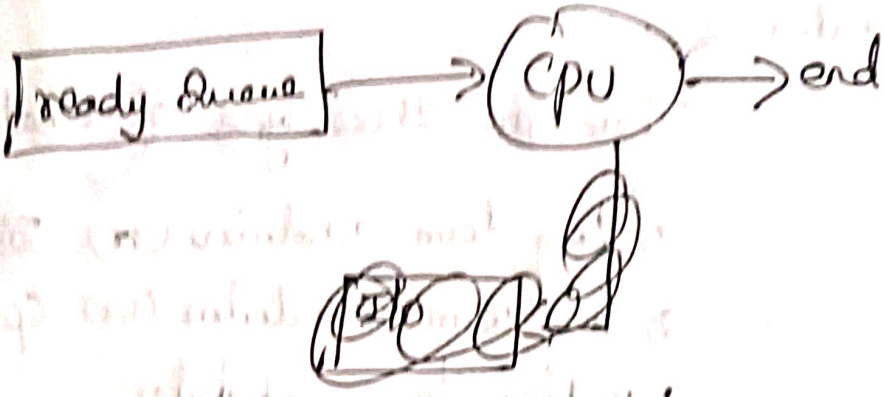
In main m/y contains Ready Queue
↳ list of processes ready for execution



It loads the process from Job Queue (ie) in secondary m/y to stores the process from Ready Queue in main m/y. This is the major work of long term scheduler.

Short term schedulers (or) CPU schedulers :-

Short term schedulers allocates frequently CPU for execution. It is frequently use Scheduler.



Types of Processes:-

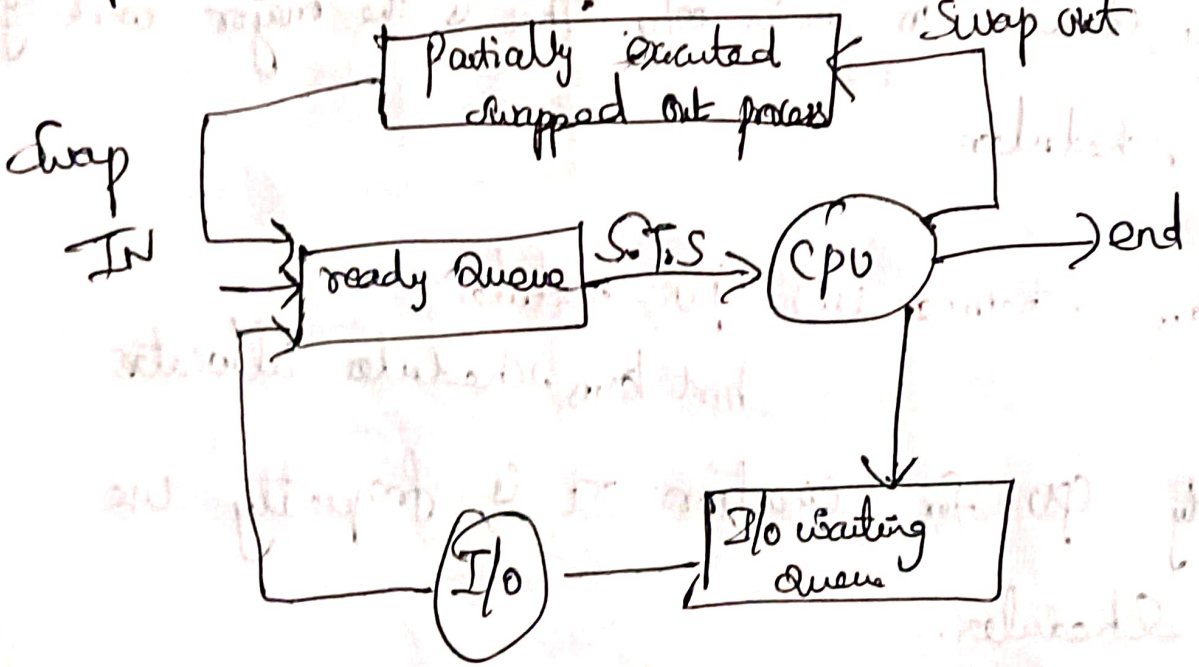
1) CPU Bounded process → (CPU instructions are more whereas I/O Instructions are less)

2) I/O Bounded process ← (I/O instructions are more where CPU instructions are less)

I/O Bounded Process used by short term Scheduler (or) CPU scheduler.

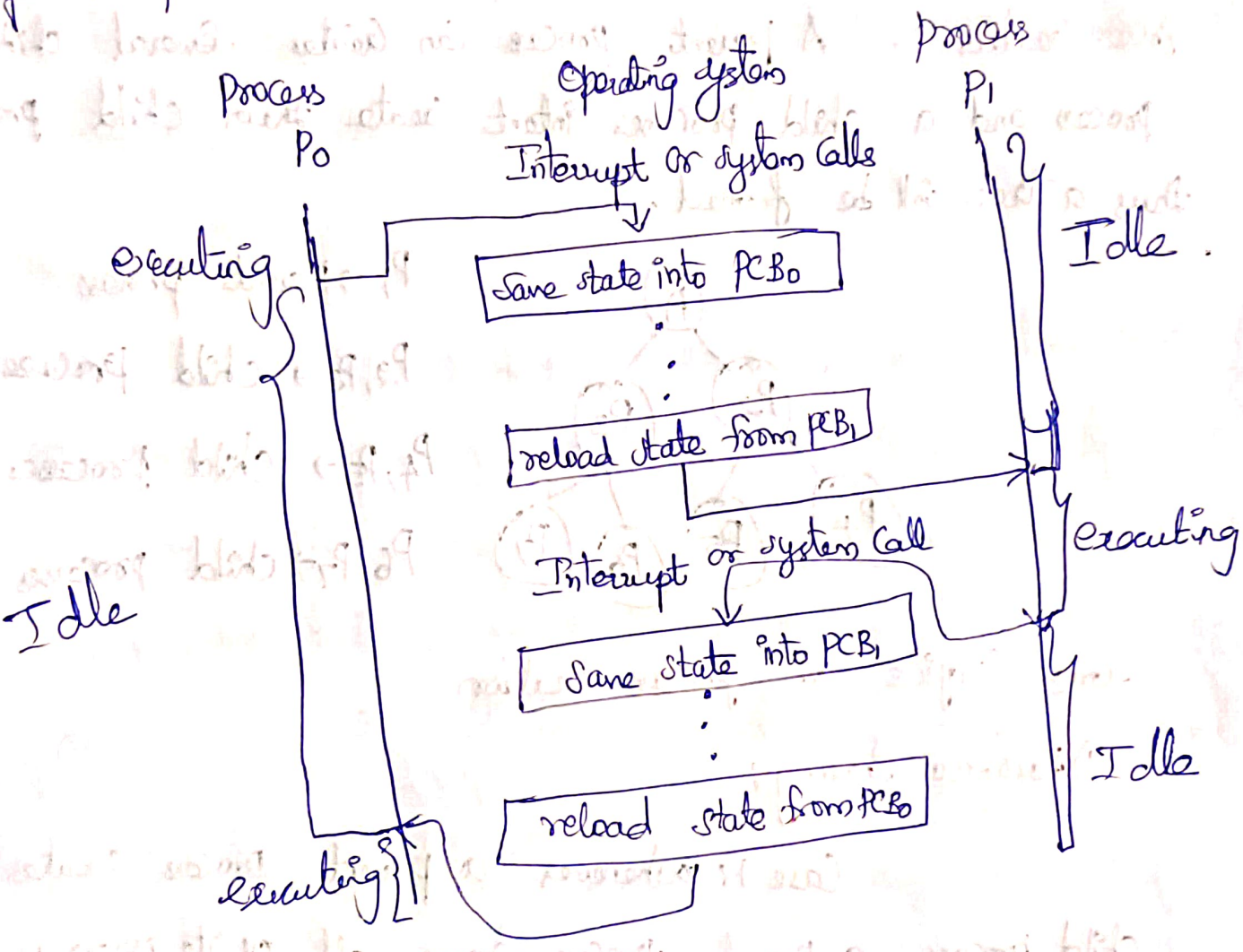
Medium term scheduler:-

It is mainly used for swapping concept.



Context switching.

CPU switches from one process to another process. (i.e) saving the state of old process and loads the state of new process.

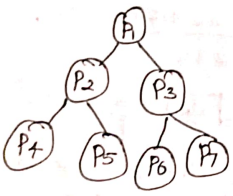


Operations on processes

There are two main operations on process

1) process Creation
2) process termination

Process Creation :- A parent process can Create several child process and a child process might Create their child processes thus a tree will be formed.



P_1 → Parent process
P_2, P_3 → child processes of P_1
P_4, P_5 → child processes of P_2
P_6, P_7 → child processes of P_3

Some Topics on process Creation

1) Resource sharing :

Case 1: whenever a parent process Creates its child process, a parent process shares all of its resources to the child process.

Case 2: Parent process shares some of its resources with child process.

Case 3: Parent process never shares resources with child process.

2) Execution :-

Case 1: Parent process and child process are executed Concurrently.

Case 2: Parent process has to wait until the child process has completed its execution. Once child process has terminated then only parent process has been terminated.

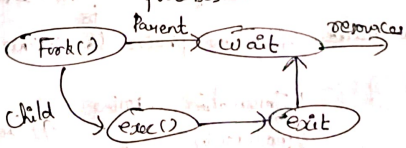
3) Address Space of a child process:

Case 1: whenever a parent process created a child process, the child process has gets exact duplicate Copy of the parent process. (i.e) address space of the child process and parent process are same but process id is different.

Case 2: Child process has a new process a loaded into it.

Examples :- 1) Fork system Call is used to Create a new child process. child process is exact duplicate of the parent process.

2) Exec system Call, then the corresponding old process is replaced with new processes.



Example Program:-

```
main()
{
 int pid;
 Pid = fork()        // used to create child process
 if (Pid < 0)  /* Error */
 {
   printf (" fork (), failed ");    // < 0 is negative no - process failed
 }
 elsif (pid == 0) /* child process */        // = 0 (ie) positive
 {                                           child process
   execlp (" /bin/ls ", " ls ", NULL);
 }
 else (pid > 0) /* parent process */          // > 0 (ie) positive
   {                                          Parent process
       wait (NULL);
       Printf (" child completed ");
   }
}
```

Inter Process Communication :-

Inter process communication mechanisms allow to communicate from one process to another process

Advantages of Ipc :-

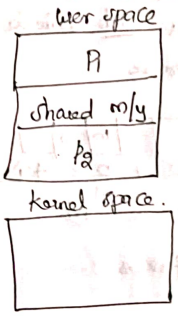
1) Information sharing
2) modularity
3) Computational speed
4) Convenience

Inter process communication mechanisms :-

There are two mechanisms in IPC

1) shared memory.
2) message passing.

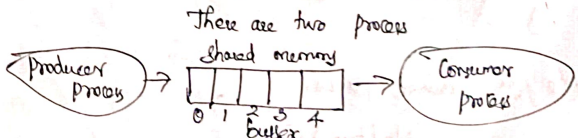
shared memory :-



In user space, we stored programs.
Eg :- P1 wants to communicate with P2 with the help of shared m/y
As well as process P2 can communicate with P1 so both the processes can perform operations like read() write() etc

Advantages :- It is very faster because it doesn't use any system calls.

Example :- Producer - Consumer Problem.



Task of the problem is producer process produces items and placed in buffer storage and consumer process

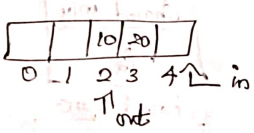
Consumes the item from buffer.

Code:- Here we can use two variables

1) in → next free position
2) out → first full position

Here in variable is used to perform operations in producer process and out variable is used to perform operations in consumer process

Eg:-



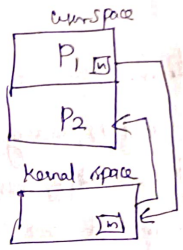
Code for Producer Process :-

```
while (true)
{
    while ((in+1)%Buffersize) == out); // Buffer is full
    buffer[in] = next produced;
    in = (in+1)%Buffersize;
}
```

Code for Consumer Process:-

```
while (true)
{
    while (in == out);
    next consumed = buffer[out];
    out = (out+1)% Buffersize;
}
```

2) Message passing :-



Here two process are there P1 & P2. Now P1 wants to send a message to P2 then P1 message is stored in kernal space. P2 wants to access the message from P1 then it is access from kernal space.

Issues in message passing :-

1) Naming
   a) Direct communication
   b) Indirect communication
2) Synchronization
   a) Blocked sender / Blocked Receiver
   b) Non-Blocked sender / Non-Blocked Receiver
3) Buffering
   a) 0 Capacity
   b) Finite Capacity
   c) Infinite Capacity.

1) Naming :-

Direct communication.

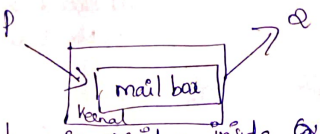
Send (Q, message)

Receive (P, message).

In the s/m call we can specify transparently (directly) the process name.

Indirect communication.

The processes cannot communicate directly. The process can be communicate with mediator. It is called mail box (or) port.



mail box is resides inside in kernal

Send (mailbox, message)

Receive (mailbox, message)

The sender and receiver can communicate through mailbox it is called inDirect communication.

2) Synchronization.

a) Blocked Sender / Blocked Receiver :- (Synchronization)

Sender sends the message, after sending the message then sender process will be blocked until receiver process receives the message. may be receiver is busy with some other operations then sender will be wait

Blocked Receiver :- The blocked receives must be blocked until the message will be received by the sender. If the sender process is busy with some other work then the receiver process has to be wait

b) Non-Blocked Sender / Non-blocked Receiver (Asynchronization)

Sender sends the messages to the receiver it won't wait for the receiver, it will starts to send the another message (i.e) it won't wait.

The receiver is busy with it owns work.

3) Buffering :- What is the size of the buffer.

a) 0 Capacity - There is no buffer (ie) the sender is wait until the receiver receives the messages.

b) Finite Capacity - The size of the buffer is limited. (ie) It is declared the size previously. may be buffer is Full then sender wait to ~~send~~ receive the message by receiver.

c) Infinite Capacity :- The buffer size is unlimited

Sockets :-

→ It is used for Communication in client-server system

→ A socket is defined as an endpoint for communication

→ A pair of Processes Communicating over a network employ a pair of sockets - one for each Process.

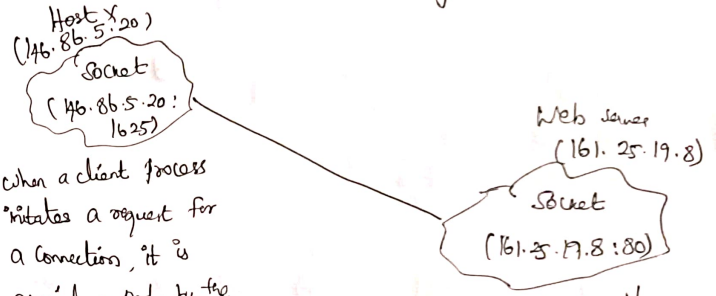
→ A socket is identified by an IP address Concatenated with a port number.

→ The server waits for incoming client request by listening to a specified port. once a request is received, the server accepts a connection from the client socket to complete the connection.

→ Server Implementing specific services (such as telnet, ftp and http) listen to well-known ports

(a telnet server listens to port 23, an ftp server listens to port 21, and a web, or http, server listens to port 80).

Communication using sockets



When a client process initiates a request for a connection, it is assigned a port by the host computer.

This port is some arbitrary number greater than 1024.

The packets traveling thru the hosts are delivered to the appropriate process based on the destination port number

Thread Libraries in OS

In order to implement thread we can use thread libraries. It provides API. API contains a collection of functions and methods which are useful for creating and managing the threads.

There are two types of threads are available.

1) pthread (POSIX)
2) Java Threads.

C language :- Calculate the sum of first 3 natural no's

Pthreads (POSIX)

```
#include <pthread.h>
#include <stdio.h>
int sum;
Void *runner(Void *param); //Void pointer stores address of any pointer.
int main(int argc, char *argv[])
{
    pthread_t tid; // thread identification number
    pthread_attr_t attr; // thread attributes (information)
    pthread_attr_init(&attr); // thread initialization
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL); // join method is used to block the main function until runner execution
    Printf("sum = %d", sum);
}
```

```
Void *runner(Void *param)
{
    int i, upper = ato i(param);
    S=0;
    for(i=1; i<=upper; i++)
        S = S+i;
    pthread_exit(0); // destroys the thread.
}
```

output :- Sum = 6.

Thread Issues in operating system :-

1) fork() and exec() system call
2) Thread Cancellation
   - Two approaches
     1) Asynchronous Cancellation
        - Killing a thread immediately
     2) Deferred Cancellation
        - Allow the thread to kill itself.
3) Signal Handling :- (special notification from one process to another process).
   To whom it will send the notification?
4) Thread pools
5) Thread specific data

Module-3

Process Synchronization Background

Eg:- producer and consumer problem.

Producer	Consumer
Count = Count+1	Count = Count-1
A: R_1 = Count	D: R_2 = Count
B: R_1 = R_1+1	E: R_2 = R_2-1
C: Count = R_1	F: Count = R_2

Count = 7, Let

A $R_1 = 7$
B $R_1 = 8$
D $R_2 = 7$
E $R_2 = 6$
C Count = 8
F Count = 6

Actually count value is 7, but here we got count value is 6, so it is called

Race Condition.

When Race condition will occur?

When multiple processes are accessible the same shared variable simultaneously and if the are execute the instructions in improper order then Race condition will occur.

When Synchronization needed :-

At a time only one process occurs and to solve race condition problem.

Critical section Problem :-

It is a part of the program where the shared memory is to be Accessed.

eg:- Producer - Consumer Problem.

In this pblm, the count variable is act as a shared variable, so count variable is a in critical section.

(i.e) count variable is incremented in producer process and the same count variable is decremented in consumer process. Both the process will be executed simultaneously then Race condition will occur.

It is called Critical section problem.

The actual Critical section must be mutual exclusion. (ie) It contains only one process at a time

How to solve Critical section problem?

we have 2 solutions to solve Critical section problem.

1) Software solution
   - ↳ Peterson's solution
2) Hardware solution
   - ↳ & synchronization H/w

Any solution to the Critical section problem, needs to satisfy three requirements

1) mutual Exclusion
2) Progress
3) Bounded waiting.

Mutual Exclusion :-

At a time only one process in the Critical section, when one process is in Critical section then other process should not allow to the Critical section

When the one process comes out from the Critical section then only the other process will allow to enter in the Critical section.

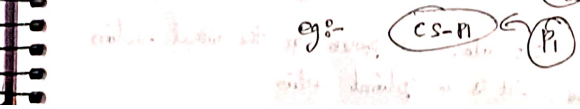
2) progress :-

In the Critical section does not have any process. and set of process $P_0, P_1, P_2 \ldots$ all are ready entering into the Critical section. Now which process will allow to the Critical section is taken infinite no. of time. (ie) decision will take more no. of time.

3) Bounded waiting :- There are 2 types of buffer. They are

1) Bounded Buffer – The size is limit
2) unbounded Buffer – The size is unlimited.

Bounded waiting means there should be bounded or limit on no of times a process can enter into a Critical section, when other process request an enter into the Critical section.

eg :- 

General structure of process Pi

```
while (true)
{
    entry section

        Critical section

    exit section

        Remainder section
}
```

In Program structure has three parts

1) Entry section
2) Critical section
3) Exit section

1) Entry section :- whenever the process checks any process is in Critical section or not. If it is free then it will enter and locks itself.
   ↳ It won't allow any other process

2) Exit section :- It unlock the process in the Critical section

3) Remainder section :- It is an optional section.